

# CineIQ

## An Explainable, Hybrid Movie Recommendation Engine

### Final Report

#### Abstract

Content discovery on modern streaming platforms is often opaque, heavily biased toward promoted titles, and traps users in repetitive recommendation loops. To address this, we developed CineIQ, an open and explainable movie recommendation engine. By combining collaborative filtering with content-based filtering, CineIQ delivers highly personalized suggestions. Furthermore, it introduces a sentiment-aware re-ranking system using Natural Language Processing (NLP) to ensure recommendations reflect genuine audience satisfaction. Finally, an explainability layer breaks down the "black box" of Machine Learning by providing users with simple, human-readable reasons for every recommendation. This report outlines the methodology, logic, and architecture behind CineIQ using intuitive concepts rather than complex mathematics.

## 1 Introduction

### 1.1 The Problem Statement

Today's streaming platforms suffer from a lack of transparency. Users are often recommended movies without understanding *why* those specific titles were chosen. These algorithms typically rely on obscure metrics, creating "recommendation loops" where users are shown the same type of content repeatedly, severely limiting content discovery. Furthermore, platforms often mix organic recommendations with promoted content, reducing trust. There is a pressing need for a recommendation engine that is transparent, personalized, and capable of evolving with the user's taste over time.

### 1.2 Project Deliverables

To solve these challenges, CineIQ was built with four main deliverables:

- **Hybrid Recommendation Engine:** A combination of user-behavior learning and content-matching to generate a broad list of good movies.
- **Sentiment-Aware Re-Ranker:** An AI reader that goes through actual user reviews to rank movies based on real audience feelings, not just star ratings.

- **User Taste Dashboard:** A clean, interactive user interface built with Streamlit.
- **Explainability Layer:** A system that tells the user exactly why a movie was recommended using natural language.

## 2 Datasets Used

A robust ML model requires rich data. CineIQ leverages three distinct datasets:

- **MovieLens 25M:** A massive dataset containing millions of user ratings. We used this to understand user behavior and preferences over time.
- **TMDB Metadata:** This dataset provides the "DNA" of the movies, including genres, keywords, production companies, and cast.
- **IMDB 50K Reviews:** A dataset containing 50,000 positive and negative movie reviews, which we used to teach our NLP model how to understand human sentiment.

## 3 System Architecture and Methodology

### 3.1 Step 1: Content-Based Filtering (TF-IDF)

The first step is to recommend movies that are similar to what the user already likes. We use a technique called **TF-IDF** (Term Frequency-Inverse Document Frequency).

Instead of looking at complex math, imagine making a "soup" of words for every movie. We combine the movie's genres, keywords, and production companies into one big paragraph. TF-IDF acts like a smart reader that figures out which words are truly unique and important. For example, the word "Action" might be common, but "Cyberpunk" is rare and specific. If a user highly rates several "Cyberpunk" movies, our system learns that preference and finds unrated movies that share those exact unique keywords.

### 3.2 Step 2: Collaborative Filtering (SVD)

While content filtering finds similar movies, it doesn't know if a movie is actually *good*. For this, we use **SVD** (Singular Value Decomposition), a collaborative filtering technique.

Think of SVD as an expert matchmaker. It looks at the massive grid of users and their movie ratings. It notices patterns—for instance, "User A and User B both loved Movie X and Movie Y." If User A then loves Movie Z, SVD predicts that User B will probably love Movie Z too. It learns hidden preferences without needing to know what the movie is actually about.

### 3.3 Step 3: The Hybrid Ensemble

Neither TF-IDF nor SVD is perfect on its own. TF-IDF might recommend a terrible movie just because it has the right keywords. SVD might recommend a popular movie that doesn't fit the user's specific niche.

CineIQ creates a **Hybrid Ensemble** by combining both scores. We normalize the scores (so they are on the same scale) and blend them together. In our implementation, we heavily weight the SVD score (80%) to ensure quality, while using the TF-IDF score to ensure relevance. This combined score gives us our **Top 50 Candidate Movies**.

### 3.4 Step 4: Sentiment-Aware Re-Ranking (DistilBERT)

Star ratings can be misleading. A movie might have a 4-star average but be highly controversial, or its recent reviews might be terrible. To fix this, CineIQ actually "reads" audience reviews.

We take our Top 50 movies and fetch their latest reviews using the TMDB API. We then pass these reviews through **DistilBERT**, an advanced AI language model created by HuggingFace. We specifically trained DistilBERT on our IMDB dataset so it knows what a happy or disappointed movie review sounds like. DistilBERT reads the reviews for the Top 50 movies and calculates a "positivity score." We use this sentiment score to re-rank the top 50, bubbling the most genuinely loved movies to the very top. This results in our final **Top 10 Recommendations**.

### 3.5 Step 5: The Explainability Layer (LIME)

The final piece of the puzzle is transparency. Users want to know why a movie is on their screen. We implemented **LIME** (Local Interpretable Model-agnostic Explanations) to provide this transparency.

LIME looks at the DistilBERT model and asks, "Which specific words in the review convinced you this was a good movie?" It extracts these exact keywords (e.g., "suspenseful," "masterpiece," "gripping"). We then use a rule-based template to combine the movie's primary genre with these keywords.

## 4 Technology Stack

To build this robust pipeline, CineIQ utilizes modern and efficient technologies:

- **Machine Learning & Data Manipulation:** Python, Pandas, NumPy, and scikit-learn for handling the massive datasets and building the TF-IDF logic.
- **Collaborative Filtering:** The Surprise library was used to train and implement the SVD algorithm.
- **Natural Language Processing (NLP):** HuggingFace's `transformers` library, specifically DistilBERT, running on PyTorch for sentiment analysis.

- **Explainability:** The `lime` library for text explainer features.
- **Frontend Application:** `Streamlit` was used to build the interactive User Taste Dashboard, allowing for a seamless and responsive user experience.
- **Experiment Tracking:** `MLflow` was integrated to track model training parameters, such as ensemble weights and SVD factors, ensuring reproducibility.
- **Data Caching:** `SQLite3` was used to cache fetched TMDb reviews, heavily reducing API calls and improving application speed.

## 5 Application Workflow

The user experience in the `Streamlit` application is designed to be frictionless:

1. **User Input:** The user enters their unique User ID or selects their favorite genres (to handle "cold start" scenarios for brand new users).
2. **Processing:** Behind the scenes, the app filters out movies the user has already seen.
3. **Scoring:** The ensemble model calculates the blended TF-IDF and SVD scores to generate the top 50 list.
4. **Re-ranking:** Cached (or freshly fetched) reviews are processed by `DistilBERT` to reorganize the list based on pure positive sentiment.
5. **Display:** The UI presents the top 10 movies, complete with titles, genres, ratings, and the LIME-generated explanation of *why* it was recommended.

## 6 Conclusion and Future Scope

CineIQ successfully demonstrates that movie recommendations do not have to be an opaque black box. By logically stacking content matching (TF-IDF), behavioral learning (SVD), and human sentiment analysis (`DistilBERT`), the platform provides highly accurate suggestions. More importantly, the integration of LIME proves that AI can communicate its reasoning to everyday users.

**Future Work could include:**

- Expanding the dashboard to include visual radar charts of user taste (e.g., tracking a user's shift from Action to Drama over decades).
- Moving the caching system from `SQLite` to an in-memory database like `Redis` for even faster real-time sentiment processing.
- Adding a "dislike" feature that dynamically updates the TF-IDF profile in real-time, instantly adjusting the recommendations.